

VLSI INTERNSHIP – TASK 2

VERILOG HDL IMPLEMENTATION OF BASIC LOGIC GATES, HALF ADDER AND FULL ADDER

Submitted By :Dhruthi S

Department of Electronics and Communication Engineering

Dayananda Sagar Academy of Technology and Management (DSATM)

Internship Domain : VLSI Design

Organization : Maincrafts Technology

June 2026

1. Introduction

Verilog HDL (Hardware Description Language) is one of the most widely used languages for designing and verifying digital circuits. It enables designers to describe hardware behavior and structure at different levels of abstraction. Verilog plays a significant role in FPGA and ASIC design flows and is extensively used in modern VLSI design.

In this task, basic logic gates, Half Adder, and Full Adder circuits were implemented using Verilog HDL. The circuits were simulated using Icarus Verilog and verified through waveform analysis using GTKWave. The simulation results were compared with the expected logic behavior to validate the correctness of the designs.

1.1 Objectives

- To understand the fundamentals of Verilog HDL.
- To implement basic logic gates using Verilog.
- To develop testbenches for digital circuit verification.
- To simulate circuits using Icarus Verilog.
- To analyze waveforms using GTKWave.
- To implement and verify Half Adder and Full Adder circuits.

1.2 Tools Used

- Verilog HDL
- Icarus Verilog
- GTKWave
- Ubuntu (WSL)

2. Basic Logic Gates Implementation

2.1 Verilog Code

The following Verilog HDL code implements the basic logic gates: AND, OR, NOT, NAND, NOR, and XOR.

logic_gates.v

```
module and_gate( input A,  input B, output Y );

assign Y = A & B;

endmodule

module or_gate( input A, input B, output Y );

assign Y = A | B;

endmodule

module not_gate(  input A,output Y);

assign Y = ~A;

endmodule

module nand_gate( input A, input B, output Y);

assign Y = ~(A & B);

endmodule

module nor_gate( input A, input B,  output Y);

assign Y = ~(A | B);

endmodule

module xor_gate( input A, input B, output Y);

assign Y = A ^ B;

endmodule
```

2.2 Testbench Code

The following testbench was developed to verify the functionality of all implemented logic gates.

tb_logic_gates.v

```
`timescale 1ns/1ps

module tb_logic_gates;

    reg A, B;

    wire and_y;

    wire or_y;

    wire not_y;

    wire nand_y;

    wire nor_y;

    wire xor_y;

    and_gate  U1 (.A(A), .B(B), .Y(and_y));
    or_gate   U2 (.A(A), .B(B), .Y(or_y));
    not_gate  U3 (.A(A),          .Y(not_y));
    nand_gate U4 (.A(A), .B(B), .Y(nand_y));
    nor_gate  U5 (.A(A), .B(B), .Y(nor_y));
    xor_gate  U6 (.A(A), .B(B), .Y(xor_y));

    initial begin

        $dumpfile("logic_gates.vcd");

        $dumpvars(0,tb_logic_gates);

        A=0; B=0; #10;

        A=0; B=1; #10;

        A=1; B=0; #10;
```

```

        A=1; B=1; #10;

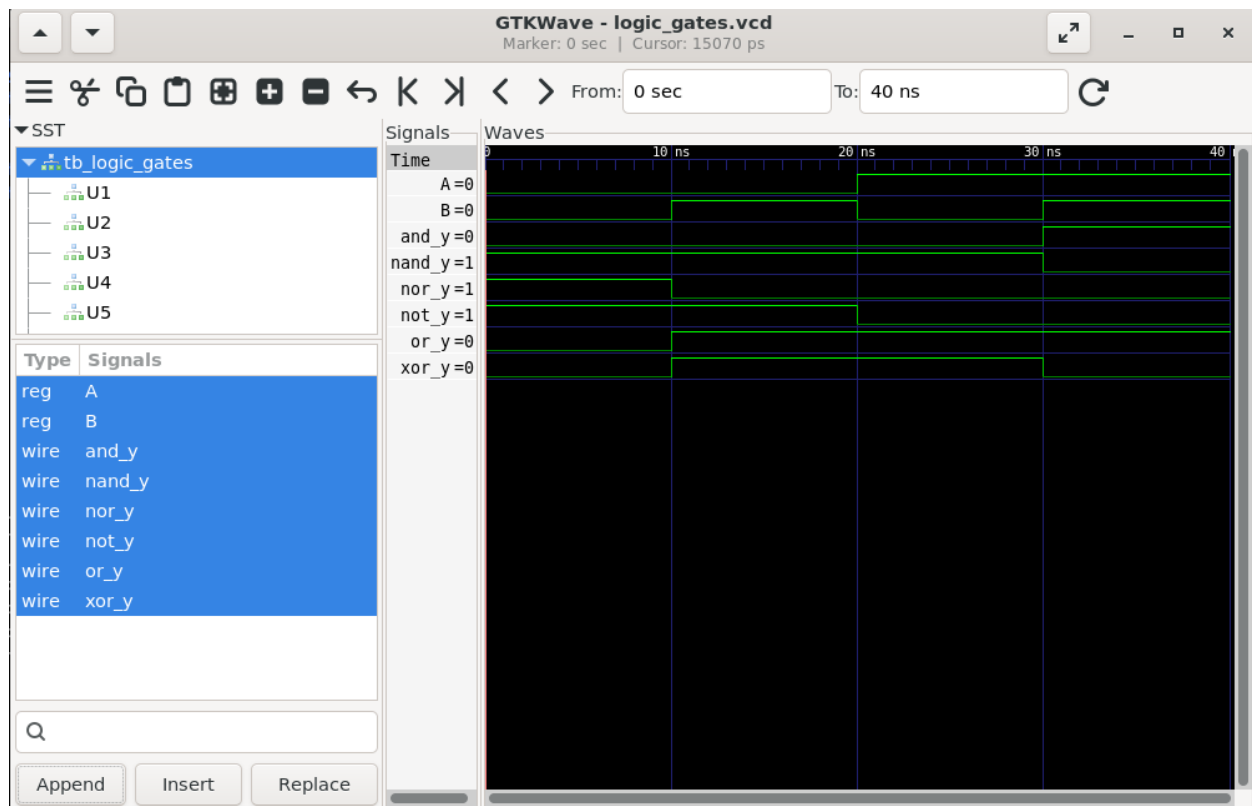
        $finish;

end

endmodule

```

2.3 Waveform Output



Observation:

The implemented logic gates were simulated successfully using Icarus Verilog and GTKWave. The waveform outputs matched the expected Boolean operations of AND, OR, NOT, NAND, NOR, and XOR gates, confirming the correctness of the design.

3. Half Adder Implementation

3.1 Verilog Code

The following Verilog HDL code implements a Half Adder using XOR and AND operations.

half_adder.v

```
module half_adder(input A, input B, output SUM, output CARRY );

assign SUM    = A ^ B;

assign CARRY = A & B;

endmodule
```

3.2 Testbench Code

The following testbench was developed to verify all possible input combinations of the Half Adder.

tb_half_adder.v

```
`timescale 1ns/1ps

module tb_half_adder;

reg A, B;

wire SUM, CARRY;

half_adder uut (.A(A), .B(B), .SUM(SUM), .CARRY(CARRY) );

initial begin

    $dumpfile("half_adder.vcd");

    $dumpvars(0, tb_half_adder);

    A=0; B=0; #10;

    A=0; B=1; #10;
```

```

        A=1; B=0; #10;

        A=1; B=1; #10;

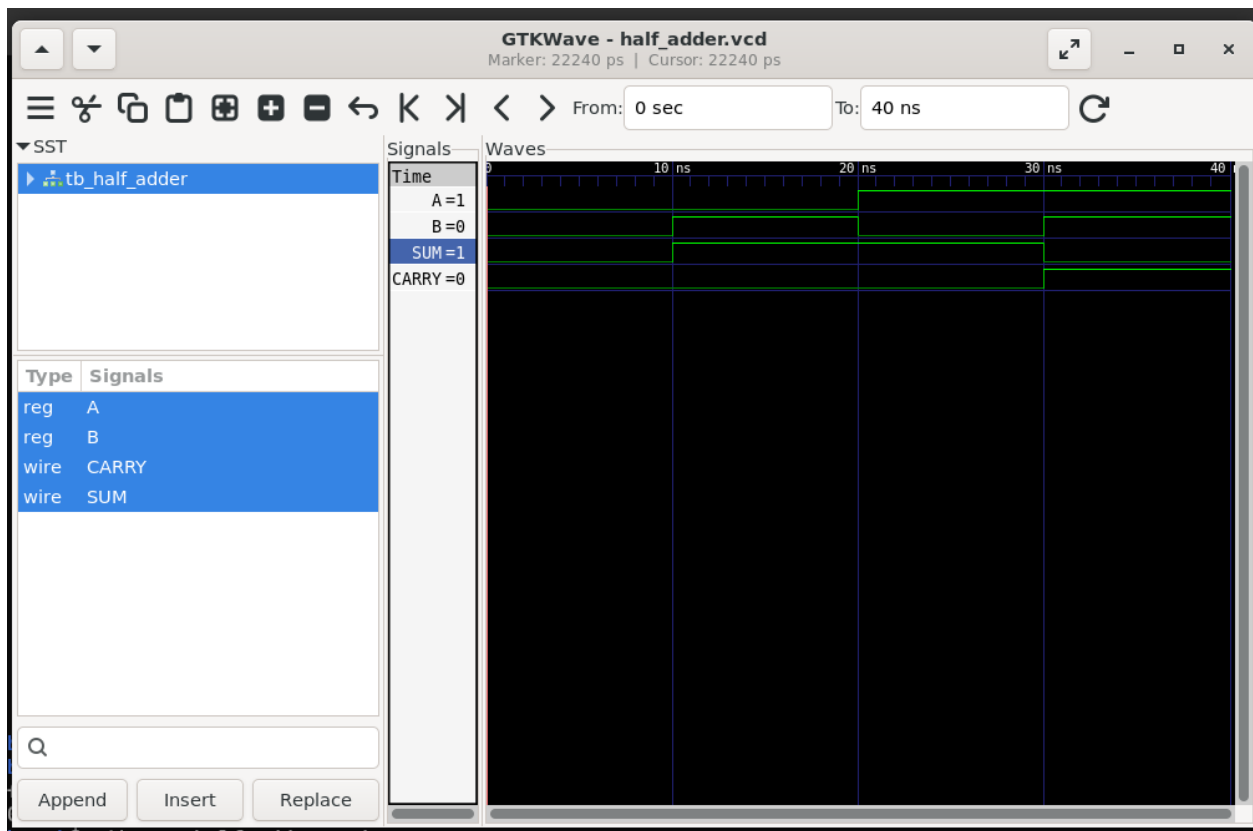
        $finish;

end

endmodule

```

3.3 Waveform Output



Observation:

The Half Adder was successfully simulated using Icarus Verilog and GTKWave. The waveform confirms that the SUM output follows XOR logic while the CARRY output follows AND logic. The outputs match the Half Adder truth table for all input combinations.

4. Full Adder Implementation

4.1 Verilog Code

The following Verilog HDL code implements a Full Adder using combinational logic.

full_adder.v

```
module full_adder(input A, input B, input Cin, output SUM,output CARRY);

assign SUM = A ^ B ^ Cin;

assign CARRY = (A & B) | (B & Cin) | (A & Cin);

endmodule
```

4.2 Testbench Code

The following testbench verifies all eight possible combinations of A, B, and Cin inputs.

tb_full_adder.v

```
`timescale 1ns/1ps

module tb_full_adder;

reg A, B, Cin;

wire SUM, CARRY;

full_adder uut (.A(A), .B(B), .Cin(Cin), .SUM(SUM), .CARRY(CARRY));

initial begin

    $dumpfile("full_adder.vcd");

    $dumpvars(0, tb_full_adder);

    A=0; B=0; Cin=0; #10;

    A=0; B=0; Cin=1; #10;

    A=0; B=1; Cin=0; #10;

    A=0; B=1; Cin=1; #10;
```



```

    A=1; B=0; Cin=0; #10;

    A=1; B=0; Cin=1; #10;

    A=1; B=1; Cin=0; #10;

    A=1; B=1; Cin=1; #10;

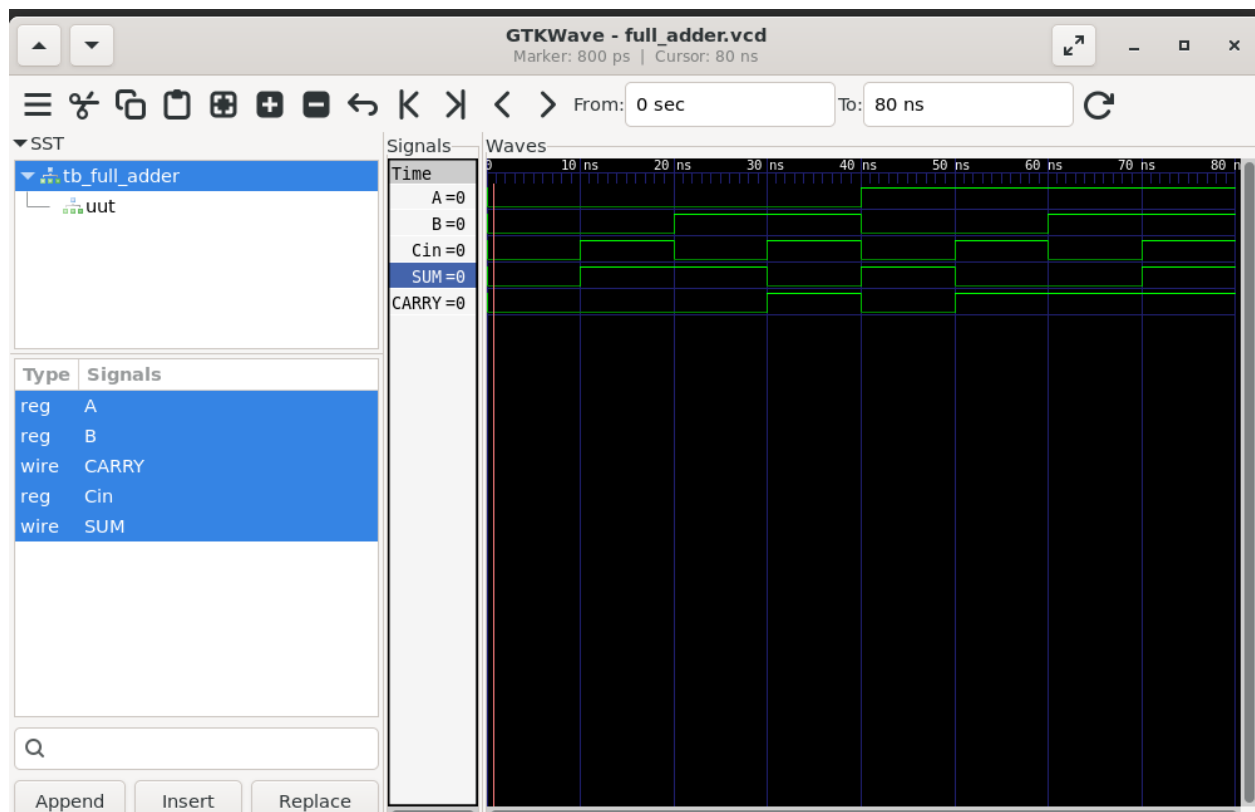
    $finish;

end

endmodule

```

4.3 Waveform Output



Observation:

The Full Adder simulation verifies correct generation of SUM and CARRY outputs for all input combinations. The waveform confirms that the design performs binary addition accurately according to the Full Adder truth table.

5. Results

The basic logic gates, Half Adder, and Full Adder circuits were successfully implemented using Verilog HDL. Functional verification was performed using testbenches, and waveform analysis was carried out using GTKWave. The outputs obtained from the simulations matched the expected Boolean expressions and truth tables.

6. Applications

- FPGA Design
- ASIC Design
- Arithmetic Logic Units (ALUs)
- Processors and Microcontrollers
- Digital Signal Processing Systems
- VLSI Design and Verification

7. Conclusion

The implementation and verification of logic gates, Half Adder, and Full Adder using Verilog HDL were successfully completed. Simulation using Icarus Verilog and waveform analysis using GTKWave confirmed the correctness of all designs. This task provided practical experience in digital circuit design, HDL coding, simulation, and verification, which are essential skills in VLSI design flow.

▪ References

1. Icarus Verilog Documentation
2. GTKWave User Guide
3. Verilog HDL – Samir Palnitkar
4. Digital Design – Morris Mano
5. IEEE Verilog HDL Standar